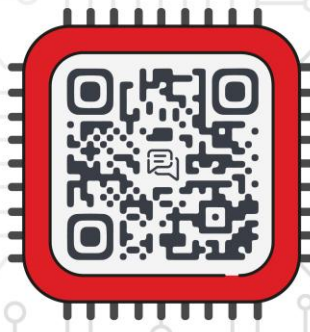




5



16



1920 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

B-Tree



د. مدحت الصوص

RB Informatics; 6/20/2025

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

لنفرض أن لدينا شجرة مكونة من مليون عقدة، في أشجار ال AVL كان تعقيد البحث عن عنصر معين هو  $\log_2 n$  أي  $\log_2 10^6 \approx 20$ .

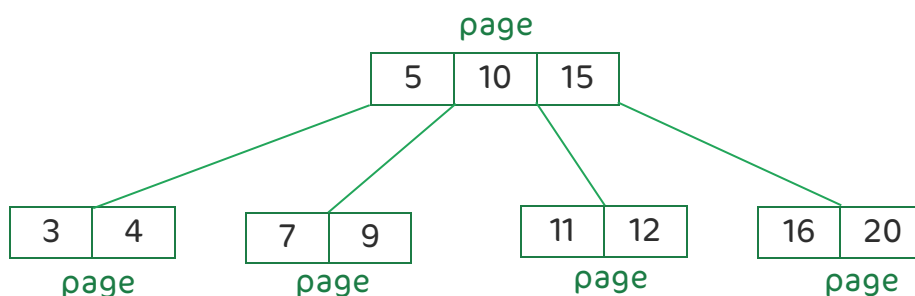
لنفرض أن لدينا شجرة أخرى مكونة من مليار عقدة ( $10^9$ ) فيكون تعقيد البحث هو  $\log_2(10^9) \approx 30$  في كلتا الحالتين التعقيد مقبول وليس بالكثير، ولكن عند البحث في الشجرة فإننا نضطر إلى نقل الشجرة بأكملها من ال hard disk إلى ال RAM، فهل من الممكن نقل مليار عقدة إلى ال RAM؟؟  
لذلك فإننا نلجأ إلى إبقاء الشجرة في ال hard disk وعند كل عملية مقارنة نذهب إلى ال hard disk لإحضار العقدة المراد المقارنة معها، وهنا ستظهر مشكلة جديدة:

عملية نقل العقدة من ال hard disk إلى ال RAM مكلفة جداً وبطيئة خاصة إذا كان التخزين من نوع HDD. فهل من الممكن تخفيف عمليات جلب البيانات من ال hard disk، أي بدلاً من أذهب 20 أو 30 مرة إلى ال hard disk هل من الممكن أن أذهب عدد مرات أقل؟  
هنا يأتي مفهوم أشجار ال B-Tree.

## B-Tree(bayer tree)

هي نوع من الأشجار المتوازنة، وهي أشجار ليست ثنائية، فبدلاً من تخزين عنصر واحد في كل عقدة، سوف نقوم بتخزين عدة عناصر فيها.

كل عقدة تحتوي على مجموعة عناصر تسمى الصفحة (page).

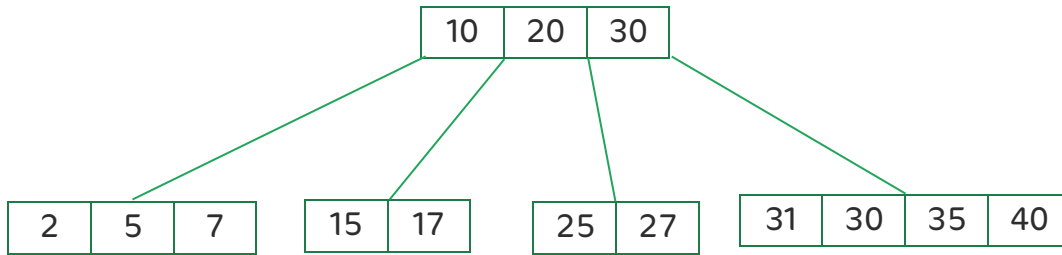


- كل شجرة يكون لها ثابت  $t$  يسمى درجة الشجرة.
- ولكن إن كنا نريد أن نطلق على شجرة أنها B-Tree من الرتبة  $t$  يجب أن تتحقق الشروط التالية:

### ⚡ شروط ال B-Tree:

1. عدد العناصر في الصفحة الواحدة يجب أن يكون محصور بين  $t$  و  $2t$ .  
أي إذا كانت  $t = 2$  فإن عدد العناصر في كل صفحة محصور بين  $[2, 4]$ .  
ولكن باستثناء الجذر، فالجذر تكون عناصره محصورة بين 1 و  $2t$ .  
أي إذا كانت  $t = 2$  فإن عدد عناصر الجذر محصورة بين  $[1, 4]$ .
2. يجب أن تكون العناصر داخل الصفحة مرتبة.
3. يجب أن تكون جميع الأوراق بنفس المستوى وهو آخر مستوى.
4. الصفحة التي تحتوي  $m$  عنصر يكون لها  $m + 1$  ابن، أي إذا كانت صفحة تحتوي على (3) عناصر فإن عدد ابناءها يكون (4) حصراً أو يكون ليس لها أبناء.

مثال: لدينا B-Tree حيث  $t = 2$ :



### ⚡ نلاحظ:

- أن هذه الشجرة تحقق جميع الشروط ال (4) السابقة، إذ أن:
- عدد العناصر في كل صفحة محصور بين  $[t, 2t]$  أي بين  $[2, 4]$ .
- الأوراق كلها في نفس المستوى.
- الجذر لديه (3) عناصر لذلك عدد أبنائه هو (4) أبناء.
- أما بالنسبة للأبناء فإن العناصر فيها تكون محصورة حسب قيم عناصر الآباء:
- ففي الشجرة السابقة، الجذر يحتوي على القيم (30) و (20) و (10)، وبالتالي كل قيم الأبناء التي تكون أصغر من ال (10) تكون موجودة على يسار ال (10)، وكل القيم الأصغر من ال (20) والأكبر من (10) تكون موجودة ما بين العنصرين (10) و (20)، وهكذا.....

## Search in B-Tree

البحث في ال B-Tree يكون على مرحلتين:

1. البحث في صفحة واحدة:

بما أننا نبحث عن عنصر معين لذلك علينا المرور بالصفحة حتى نجد العنصر، ويمكن أن نبحث في الصفحة إما باستخدام البحث الثنائي أو باستخدام البحث الخطي. وإن لم نجد العنصر ننتقل إلى الصفحة التالية.

2. الانتقال للصفحة التالية:

إذا لم نجد العنصر فسيكون العنصر كالاتي:

- إما أكبر من كل العناصر في الصفحة التي نبحث فيها.

- أو أصغر من العناصر.

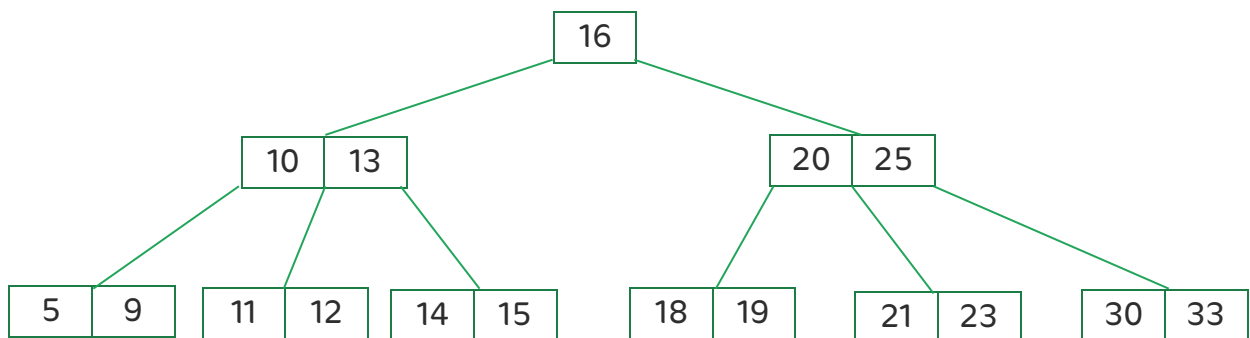
- أو موجود بين قيمة عنصرين في الصفحة.

وحسب الحالات السابقة ننتقل إلى الصفحة التالية.

### توضيح:

البحث يبدأ من ال Root، نقارن معه وحسب القيمة ننتقل إلى الصفحة التالية ونبحث داخلها عن هذا العنصر، فإذا نجده في الصفحة الحالية أو ننتقل إلى الصفحة التالية.

مثال: لتكن لدينا الشجرة التالية:



البحث عن العدد (21):

نقارن مع الجذر ونجد أن  $16 < 21$  لذلك نذهب إلى يمين الجذر.

الصفحة على يمين الجذر هي 

|    |    |
|----|----|
| 20 | 25 |
|----|----|

 ، نبحث في هذه الصفحة عن العنصر (21) ونجد أن العنصر غير

موجود ولكن  $20 < 21 < 25$  لذلك نذهب إلى الابن الذي بين (25) وال (20).

الصفحة التي بين ال (20) وال (25) هي 

|    |    |
|----|----|
| 21 | 23 |
|----|----|

 وعند البحث في هذه الصفحة نجد العنصر (21).

## تعقيد البحث



في الواقع التعقيد يعتمد على:

للم العمليات ضمن الصفحة:

الصفحة فيها من  $t \leftarrow 2t$  عقدة، في أسوأ الأحوال  $2t$  إذا البحث:

إما بحث خطي (linear search)  $2t \leftarrow$  إذا  $O(t)$ .

أو بحث ثنائي (binary search)  $\log(2t) \leftarrow$  إذا  $O(\log(t))$ .

للم عدد الصفحات:

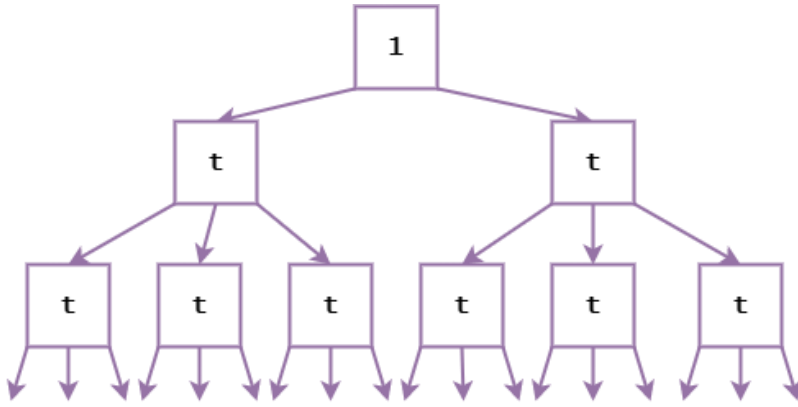
عدد الصفحات التي سنبعث فيها: في أسوأ الأحوال يكون العنصر غير موجود أو في آخر صفحة إذا  $O(h)$ .

التعقيد الكلي هو:  $O(h \cdot \log(t))$  أو  $O(h \cdot t)$

لكننا نريد التعقيد بدلالة  $n$  وليس  $h$  لذا سنقوم بإيجاد علاقة بين  $n$  و  $h$ .

سنهتكم بأسوأ حالة ( $h$  أكبر ما يمكن) حيث لدينا  $n$  عنصر ونريد توزيعه على Tree-B بأطول ارتفاع بالتالي نضع في

كل Page أقل عدد ممكن، والآن سنبدأ بالشجرة مستوى تلو الآخر.



| الارتفاع | عدد الصفحات    | عدد العناصر بالمستوى |
|----------|----------------|----------------------|
| 0        | 1              | 1                    |
| 1        | 2              | $2t$                 |
| 2        | $2(t+1)$       | $2t(t+1)$            |
| 3        | $2(t+1)^2$     | $2t(t+1)^2$          |
| ....     | ....           | ....                 |
| $h$      | $2(t+1)^{h-1}$ | $2t(t+1)^{h-1}$      |

⚡ نلاحظ من الجدول وجود نمط معين!!

■ لحساب  $n$  عدد العناصر الكلي يكون مجموع عدد العناصر في كل المستويات أي هو sum للعمود الأخير في

الجدول السابق.

■  $h$  يعبر هنا عن عدد المستويات.

في أسوأ حالة يكون ارتفاع الشجرة أكبر ما يمكن و كل مستوى يحتوي على الحد الأدنى المسموح به من العناصر  
نريد إيجاد العلاقة بين  $n$  و  $h$ ، فتكون في الحالة العامة:

$$n \geq 1 + 2t + 2t(t + 1) + 2t(t + 1)^2 + \dots + 2t(t + 1)^{h-1}$$

بينما في أسوأ حالة تكون :

$$n = 1 + 2t + 2t(t + 1) + 2t(t + 1)^2 + \dots + 2t(t + 1)^{h-1}$$

إذاً  $n$  بأسوأ الحالات تساوي هذا المقدار:

$$n = 1 + t \sum_{i=1}^h 2(t + 1)^{i-1} \Rightarrow n = 1 + 2t \left( \frac{(t + 1)^h - 1}{(t + 1) - 1} \right)$$

متتالية هندسية

$$n = 1 + 2(t + 1)^h - 2 \Rightarrow \frac{n + 1}{2} = (t + 1)^h$$

نأخذ لوغاريتم الطرفين:

$$\log_{t+1} \left( \frac{n + 1}{2} \right) = h$$

يمكننا القول إن  $h \approx \log_t n$

بأسوأ الحالات كان التعقيد:  $O(h \cdot \log(t))$  أو  $O(h \cdot t)$

أصبح التعقيد:  $O(\log(t) \cdot \log_t(n))$  أو  $O(t \cdot \log_t(n))$

تعمل  $t$  أو  $\log(t)$  لأنها ثوابت فيكون التعقيد النهائي:  $O(\log_t(n))$

ممکن أن تختلف هذه القيم في المراجع حسب تعريف  $t$  وحسب عدد المستويات لكن ما يهمنا هو فهم الاستنتاج بشكل عام لمعرفة التعقيد النهائي.

## Insert in B-Tree

الإضافة هنا تكون حصراً في الأوراق، ونميز حالتين:

1. الإضافة إلى صفحة مع عدم اختلال أحد الشروط:

نعلم بأن أول شرط ل B-Tree هو أن يكون عدد العناصر داخل الصفحة محصور بين  $[t, 2t]$ ، فإذا أضفنا عنصراً ومازالت الصفحة المضاف إليها تحقق الشرط لا نعدل شيئاً على الشجرة (طبعاً الجذر فيه استثناء وهو أن عدد عناصره محصورة بين  $[1, 2t]$ ).

## 2. الإضافة إلى صفحة مع اختلال أحد الشروط:

إن كان لدينا شجرة درجتها  $t = 2$  والصفحة في المستوى الأخير تحتوي على (4) عناصر، فعند الإضافة لهذه الصفحة سيصبح عدد العناصر (5) وهو أكبر من  $2t = 4$  لذلك سيختل الشرط الأول، ولحل هذه المشكلة نطبق الخطوات التالية.

## خطوات إضافة عنصر:

- بما أن الحد الأقصى لعدد العناصر في كل صفحة هو عدد زوجي (لأننا نضرب ب (2) أي  $2t$ ) فإنه عند إضافة عنصر زائد سوف يكون حجم المصفوفة دائماً فردي.

## مثال لتوضيح الفكرة:

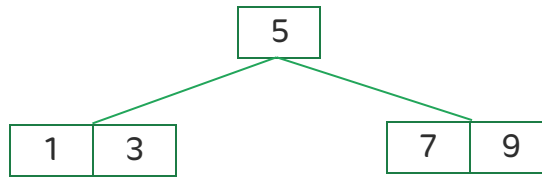
لنفرض أن الصفحة 

|   |   |   |   |
|---|---|---|---|
| 1 | 3 | 5 | 7 |
|---|---|---|---|

 موجودة بشجرة درجتها  $t = 2$ ، أي أن هذه الصفحة تحتوي على أقصى عدد من العناصر فيها، وهو عدد زوجي بالتأكيد لأننا نضرب ال  $t$  ب (2).  
إذا أضفنا العدد (9) لهذه الصفحة ستصبح 

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 3 | 5 | 7 | 9 |
|---|---|---|---|---|

 أصبح حجم المصفوفة (5)، وهو أكبر من الحد المسموح به، ونلاحظ أن حجم المصفوفة أصبح فردياً، لذلك نأخذ العنصر الأوسط ونرفعه إلى المستوى الأعلى منه.



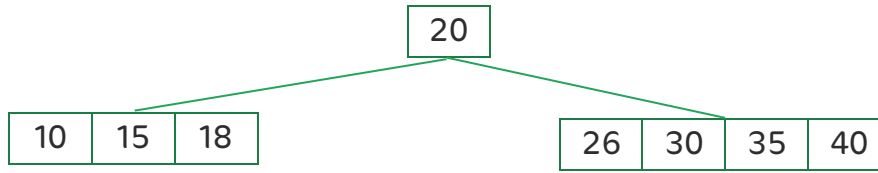
نلاحظ أننا جعلنا العنصر الأوسط هو الأب وقسمنا الصفحة إلى صفتين، الصفحة اليسارية تحتوي على القيم الأصغر من (5)، والصفحة اليمينية تحتوي على القيم الأكبر من (5)، وبهذه الطريقة نضيف جميع العناصر إلى الشجرة.

ولكن ماذا لو كان الأب يحتوي على  $2t$  عنصر ولا يمكن رفع العدد الأوسط إليه؟

في هذه الحالة سوف نكرر عملية الرفع للأب، وإذا واجه الأب نفس المشكلة نكرر عملية الرفع حتى نصل إلى الجذر، فإذا كان الجذر أيضاً يحتوي على  $2t$  عنصر عندها نفتح مستوى جديد فوق الجذر ونجعل العنصر الأوسط هو الجذر الجديد. (ولهذا السبب هناك استثناء للجذر ليكون عدد قيمه محصور بين  $[1, 2t]$ ).

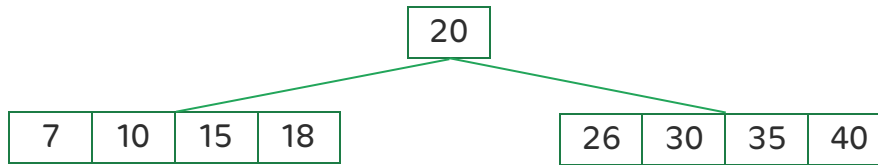


مثال: ليكن لدينا الشجرة التالية من الرتبة  $t = 2$  :



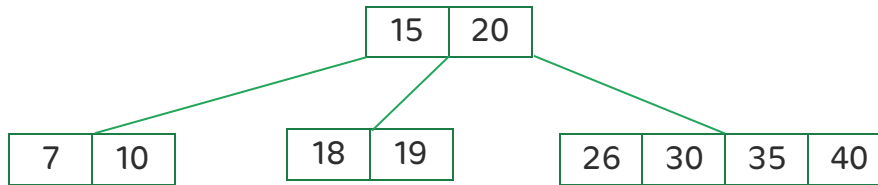
#### ■ إضافة الرقم (7):

- أولاً نبحث عن المكان الملائم للرقم (7).
- نلاحظ أن (7) أصغر من (20) لذلك نذهب إلى اليسار.
- العنصر (7) هو أصغر عنصر في الصفحة لذلك يوضع كأول عنصر، فتصبح الشجرة بالشكل:



#### ■ إضافة الرقم (19):

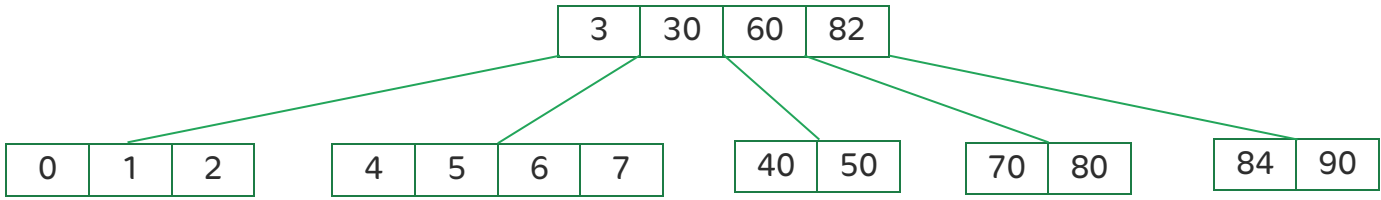
- أولاً نجد المكان الملائم، وهو على يسار الجذر.
- بعد الإضافة أصبحت الصفحة اليسارية تحتوي على عناصر أكثر من  $2t$  وهنا اختل شرط الشجرة، لذلك نأخذ العنصر الأوسط ونرفعه إلى الأب ومن ثم نقسم الصفحة إلى قسمين، العناصر الأكبر من العنصر الأوسط تكون موصولة إلى يمينه والعناصر الأصغر منه تكون موصولة إلى يساره.



■ ملاحظة: في المثال السابق، بما أن الجذر أصبح فيه عنصرين، لذلك صار لديه (3) أبناء وذلك طبقاً للشرط الرابع في ال B-Tree.

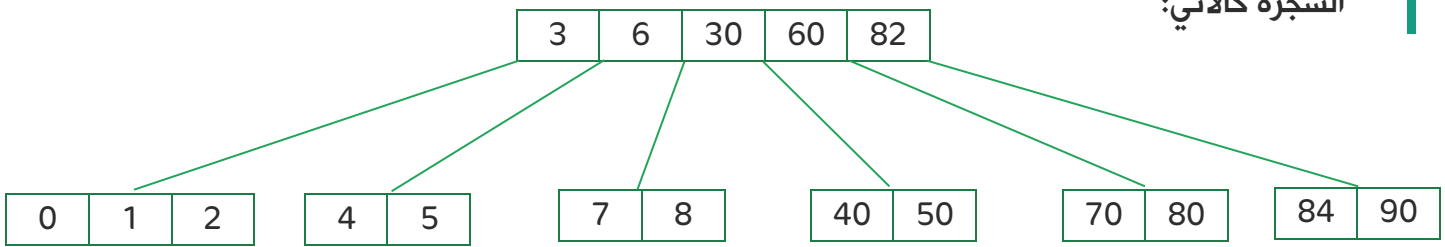


مثال: لتكن لدينا الشجرة التالية من الرتبة  $t = 2$ :

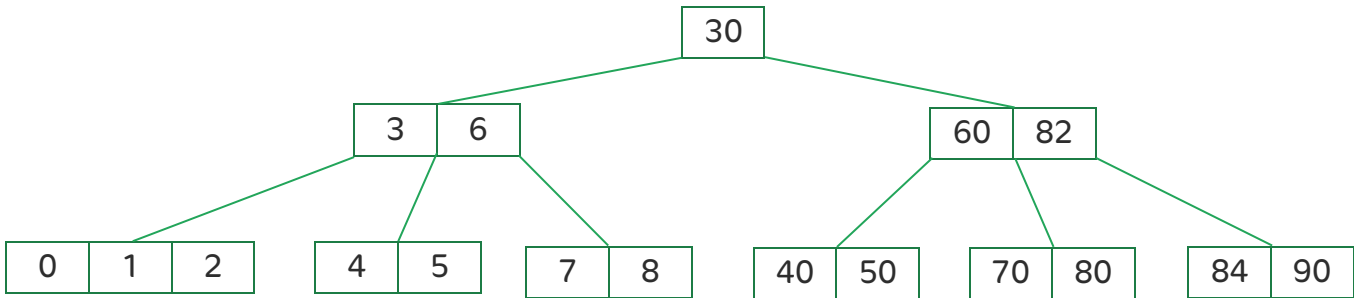


■ إضافة الرقم (8):

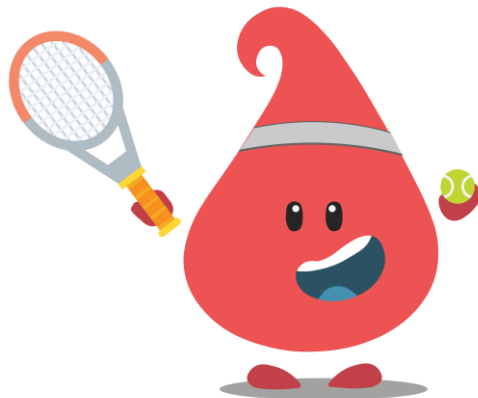
- نبحث أولاً عن المكان المناسب للعنصر، فنجد أن  $3 < 8 < 30$  لذلك تكون الصفحة ما بين (3) و (30).
- أصبح في الصفحة عدد أكبر من  $2t$  لذلك نرفع العنصر الأوسط للجذر وتصبح الشجرة كالآتي:



- ولكن أصبح حجم الأب أكبر من  $2t$  لذلك نكرر عملية الرفع مرة أخرى، وبما أن الأب هو الجذر فسوف نفتح مستوى جديد نضع فيه العنصر الأوسط، فتصبح الشجرة كالآتي:



نلاحظ أن الشجرة تحقق جميع شروط ال B-Tree.



## تعقيد الإضافة

بما أننا نبحث عن المكان الملائم لكل عقدة مضافة، فإن التعقيد يكون من نفس تعقيد البحث وهو:

$$O(\log_t(n)) \text{ أو } O(t \cdot \log_t(n))$$

ومضافاً إليه  $O(t)$  لأننا عند الإضافة نقوم بالإضافة في مصفوفة (من الممكن أن تكون  $O(1)$  إذا استخدمنا Linked list بدلاً من المصفوفة العادية).

- كلفة الإصلاح:

أسوأ الأحوال أن نضطر للإصلاح لجميع الصفحات حتى نصل إلى الجذر، إذ نقوم ب  $(h)$  إصلاح أو  $\log_t(n)$  وبعدها تعقيد الإضافة للصفحة الأب كما سبق أي  $O(t)$ .

ويصبح التعقيد النهائي من رتبة:

$$O(\log_t(n))$$

## الحذف Deletion

نميز حالتين للحذف:

حذف ورقة أي في المستوى الأخير. حذف من صفحة داخلية.

أولاً: الحذف من الورقة:

- الصفحة فيها أكثر من  $t$  عنصر: نحذف بكل بساطة.

- الصفحة فيها  $t$  عنصر: بهذه الحالة سيصبح بالصفحة أقل من  $t$  عنصر بالتالي اختل شرط B-Tree، ما الحل؟؟!

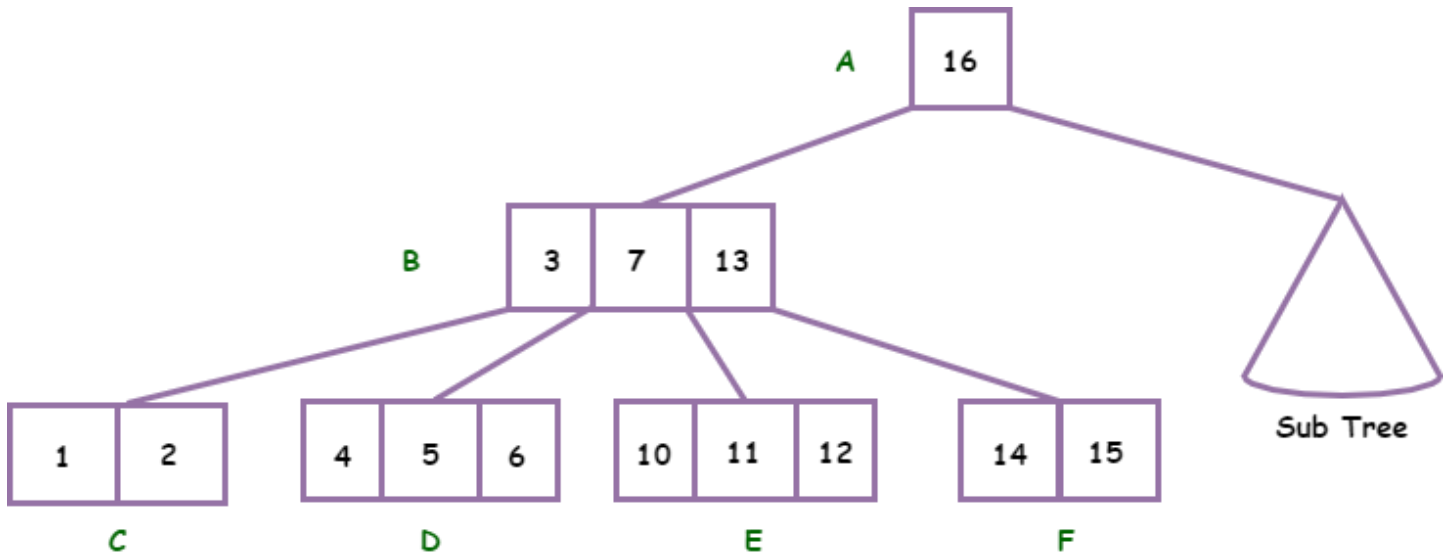
نقوم باستعارة عنصر من الأب فيصبح عدد العناصر في الصفحة الابن  $t$  عنصر، ولكن سنواجه مشكلة أخرى:

نعلم أن كل صفحة تحوي  $m$  عنصر ولها  $m + 1$  ابن وعند استعارة عنصر من الأب ستقل  $m$  بمقدار واحد وعدد الأبناء سيبقى  $m + 1$  ولحل هذه المشكلة سنميز حالتين:

الحالة الأولى: إذا كان للصفحة الابن أخ يحوي أكثر من  $t$  عنصر سيقوم الأخ بتعويض الأب عن العنصر الذي فقده.

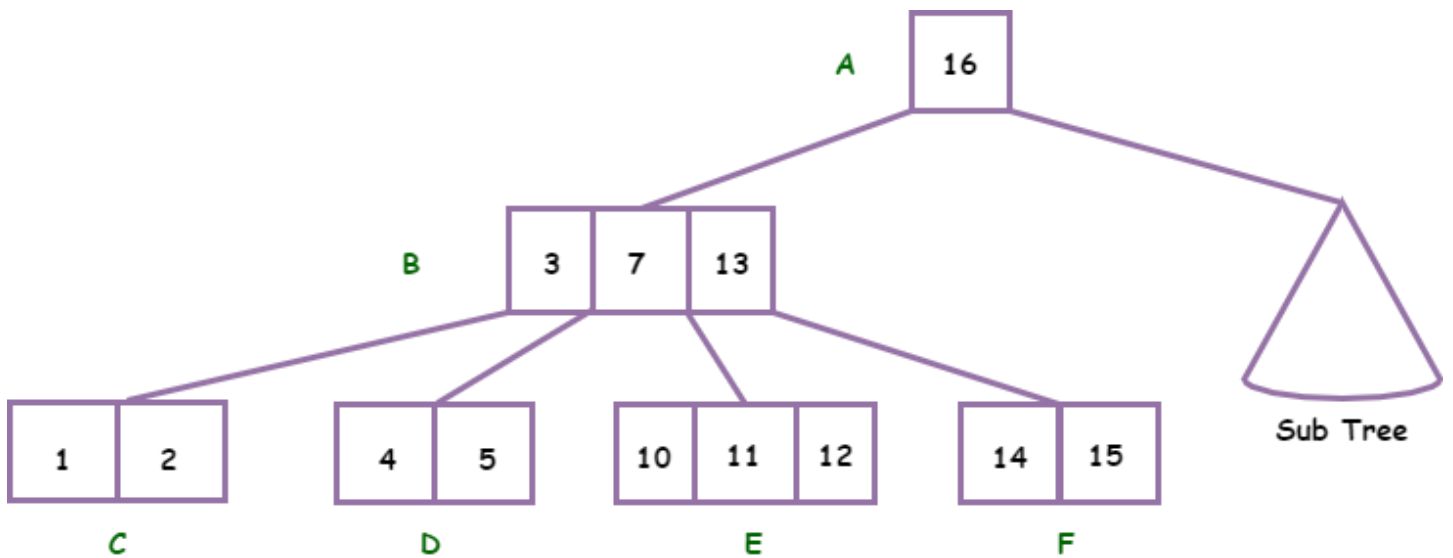
الحالة الثانية: إذا كان للصفحة الابن أخ يحوي  $t$  عنصر ندمج صفحة الابن مع الأخ فالابن فيها  $t$  عنصر والأخ فيها  $t$  عنصر إذاً الصفحة الجديدة تحوي  $2t$  عنصر ولم يخل شرط الشجرة.

مثال توضيحي: لدينا B-Tree التالية حيث  $t = 2$ :



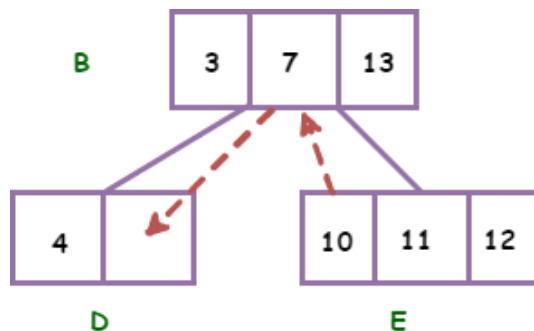
نريد حذف العنصر 6:

العنصر 6 موجود في الصفحة D وهو ورقة تحوي أكثر من  $t$  عنصر  $\leftarrow$  نحذف العنصر ببساطة دون أي تغيير فتصبح:

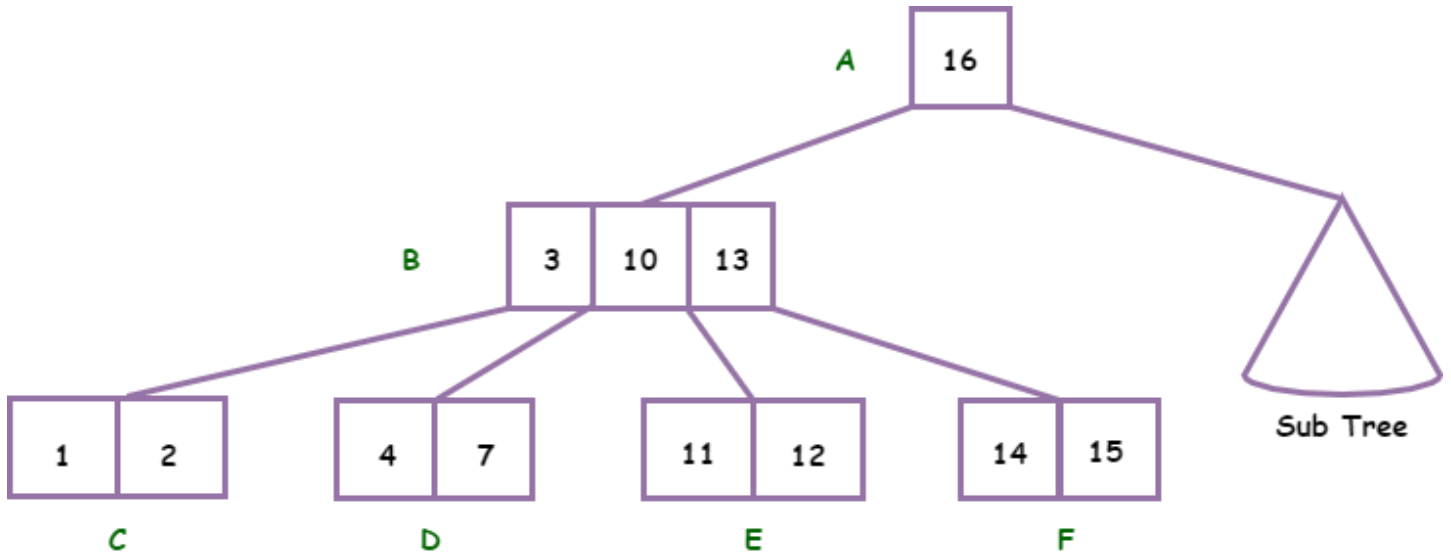


نريد حذف العنصر 5:

العنصر 5 موجود في الصفحة D وهي ورقة ولكن تحوي  $t$  عنصر  $\leftarrow$  بالتالي أخ الصفحة D هو E يحوي أكثر من  $t$  عنصر فيمكن التعويض عن طريق الأخ  $\leftarrow$  نضع مكان الـ 5 عنصر من الأب وسيقوم الأخ بتعويض الأب عن العنصر المفقود.

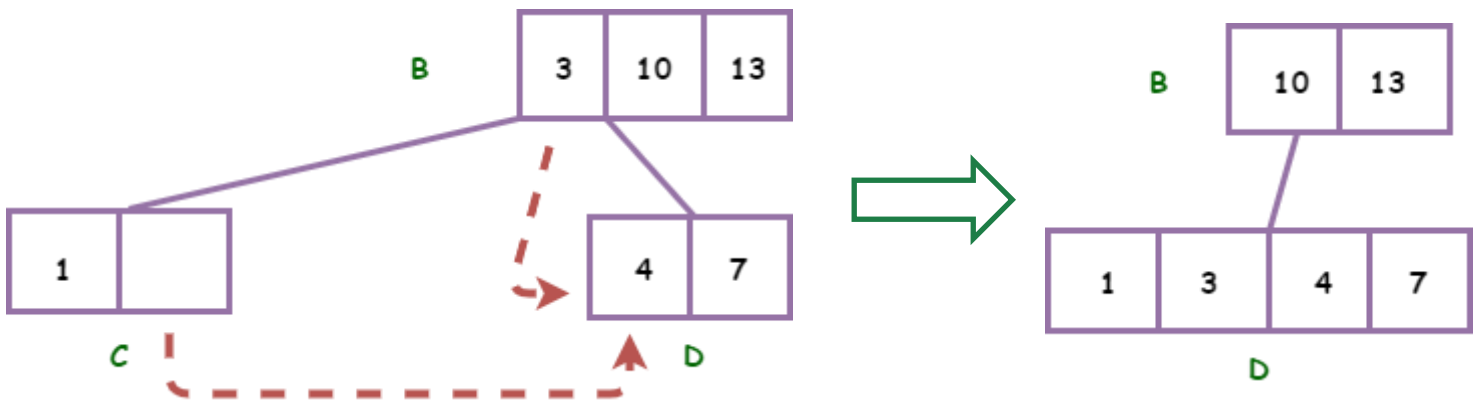


يصبح الشكل النهائي:

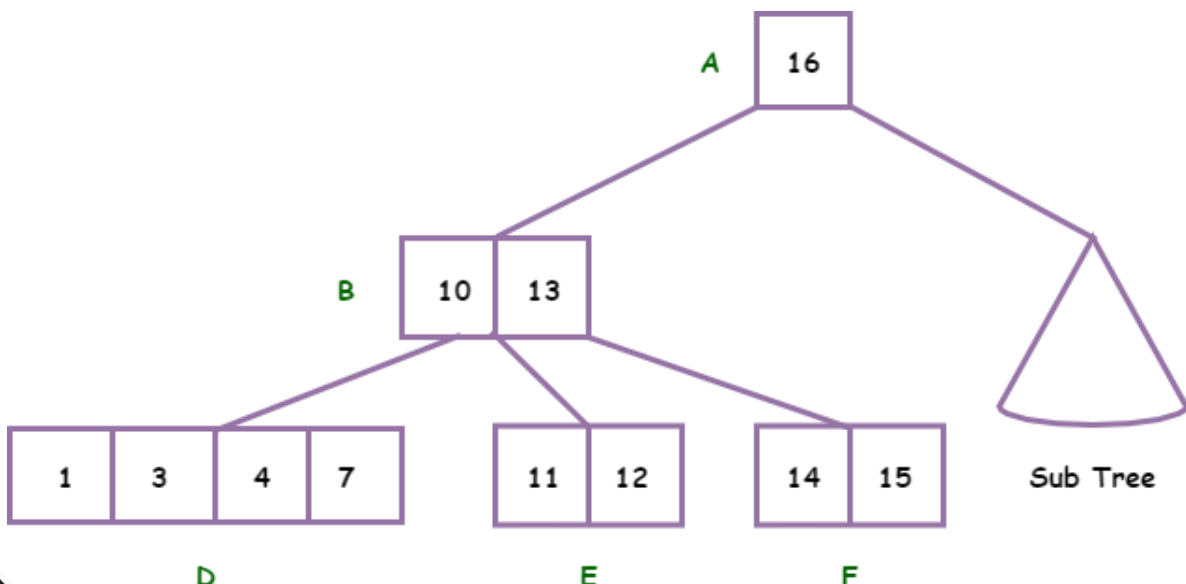


نريد حذف العنصر 2:

العنصر 2 موجود في الصفحة C وهي ورقة ولكن تحوي على t عنصر ← بالتالي أخ الصفحة C هو D ولكن لا يحوي أكثر من t عنصر بالتالي سنحذف العنصر 2 ندمج العنصر 1 المتبقي مع الأخ D بالإضافة لعنصر ننزله من الأب.



يصبح الشكل النهائي:

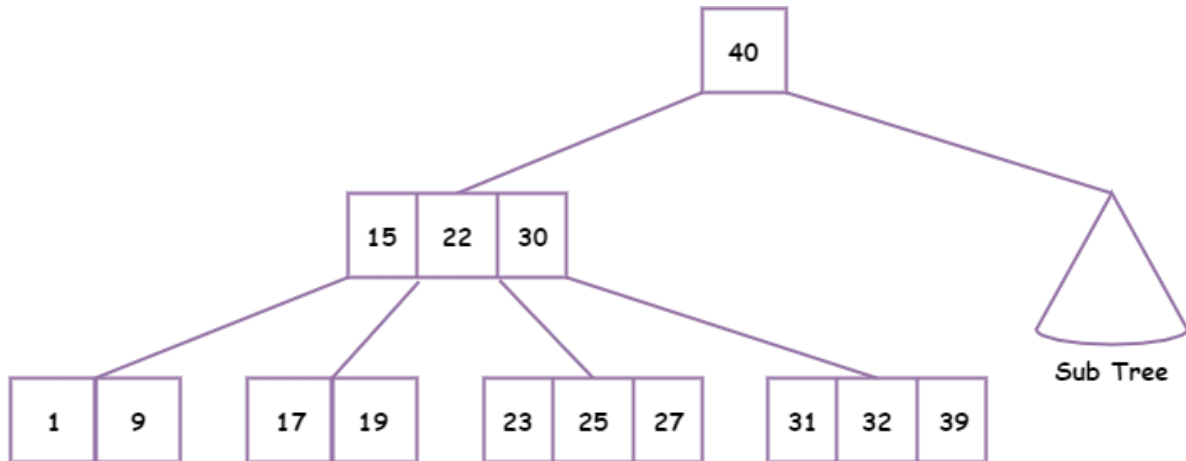


ثانياً: الحذف من صفحة داخلية:

وهنا سنميز أربع حالات مرتبة حيث إذا لم نتحقق الأولى نلجأ للثانية وإذا لم نتحقق الثانية نلجأ للثالثة وهكذا ...

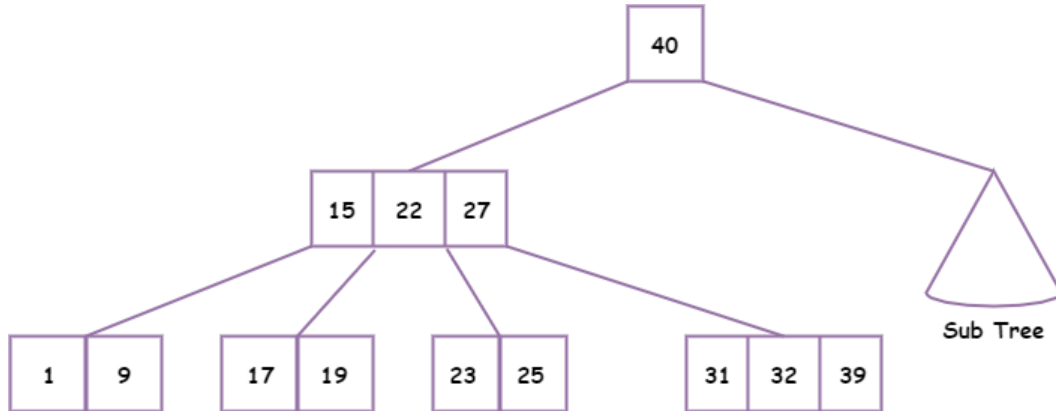
الحالة الأولى: العنصر الذي نريد حذفه يملك عنصر تالي أو سابق أحدهما قابل للحذف، نقوم بفحص إمكانية حذف العنصر السابق ثم العنصر التالي ثم نبدل الممكن منهم مع العنصر المراد حذفه ويتم الحذف.

مثال توضيحي: لدينا B-Tree التالية حيث  $t = 2$ :



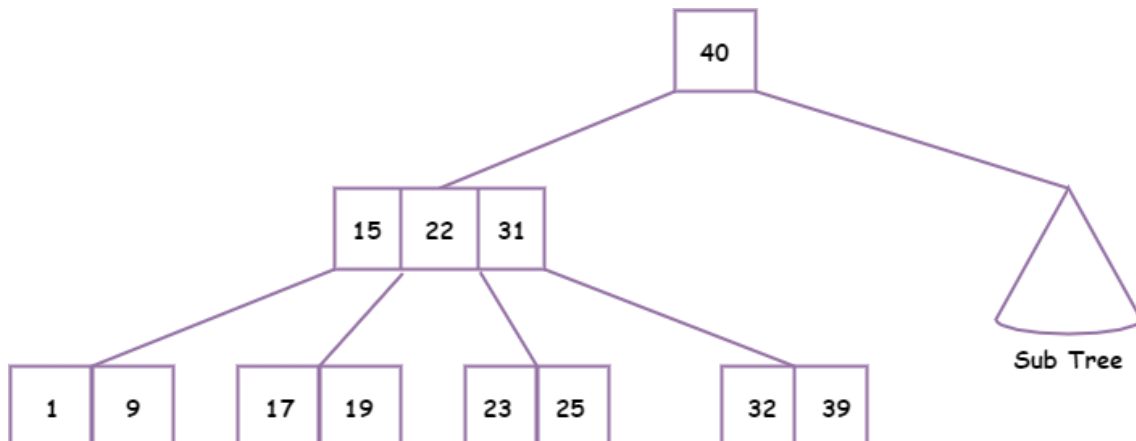
نريد حذف العنصر 30:

في الصفحة الداخلية نجد أن له عنصر سابق هو 27 نبدل ال 30 مع ال 27 ونحذف ال 30 بالتالي:



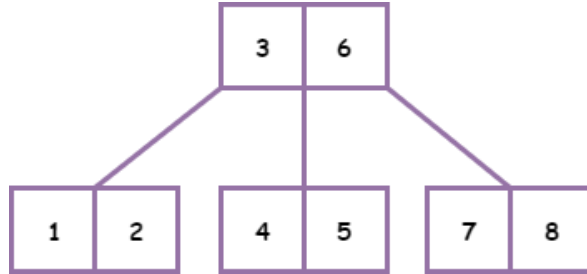
نريد حذف العنصر 27:

العنصر 27 في الصفحة ليس له عقدة سابقة قابلة للحذف لكن له عنصر تالي قابل للحذف وهو 31 نبدله مع 27 ونحذف العنصر 27:

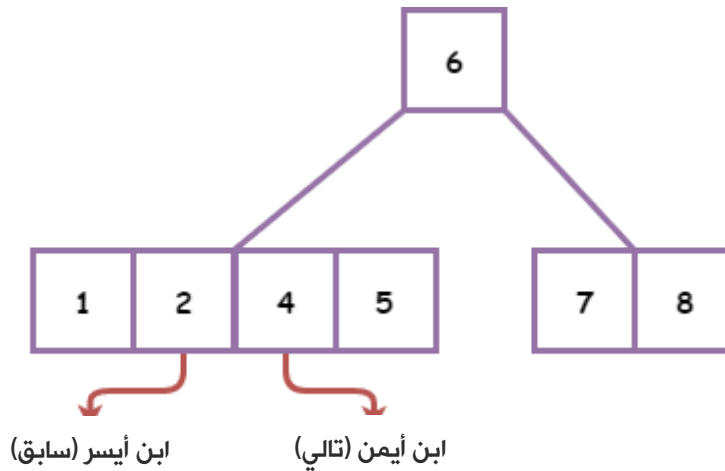


**الحالة الثانية:** الصفحتان التالية والسابقة للعنصر المراد حذفه تملكان  $t$  عنصر أي غير قابلتين للحذف لذا سنقوم بحذف العنصر ودمج ابنيه معاً وذلك في حالة كانت صفحة العنصر تحوي أكثر من  $t$  عنصر أو كانت تحوي  $t$  عنصر ولكنها في صفحة الRoot.

**مثال توضيحي:** لدينا B-Tree التالية حيث  $t = 2$ :

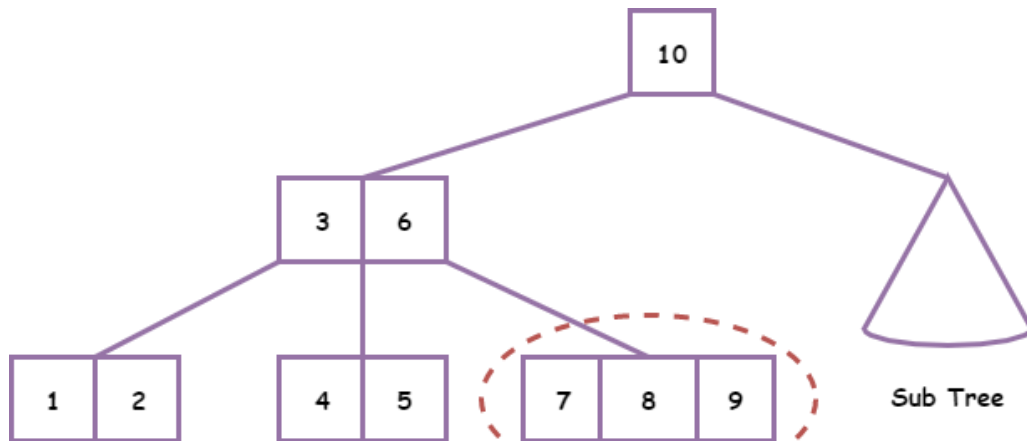


نريد حذف العنصر 3 والعنصر 3 صفحة داخلية وابنها التالي والسابق لا يمكن تبديلهما "غير قابلين للحذف" وبالتالي سندمج بناءً على الحالة الثانية



**الحالة الثالثة:** صفحة العنصر الذي نريد حذفه أحد أبنائها يملك أكثر من  $t$  فنقوم باستعارة عنصر من هذه الصفحة مع المحافظة على ترتيب العناصر (الابن الأيسر هو الأصغر والأيمن هو الأكبر) وذلك من خلال عمليات تبديل وإزاحة.

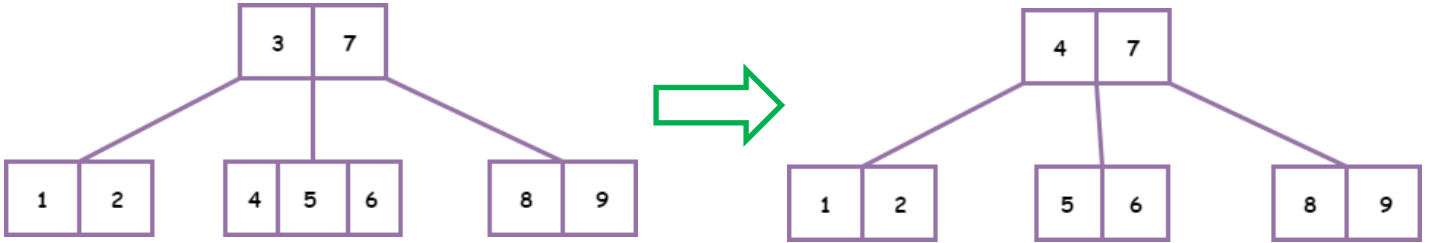
**مثال توضيحي:** لدينا B-Tree التالية حيث  $t = 2$ :



أحد أبنائها يملك أكثر من  $t$  عنصر

نريد حذف العنصر 3:

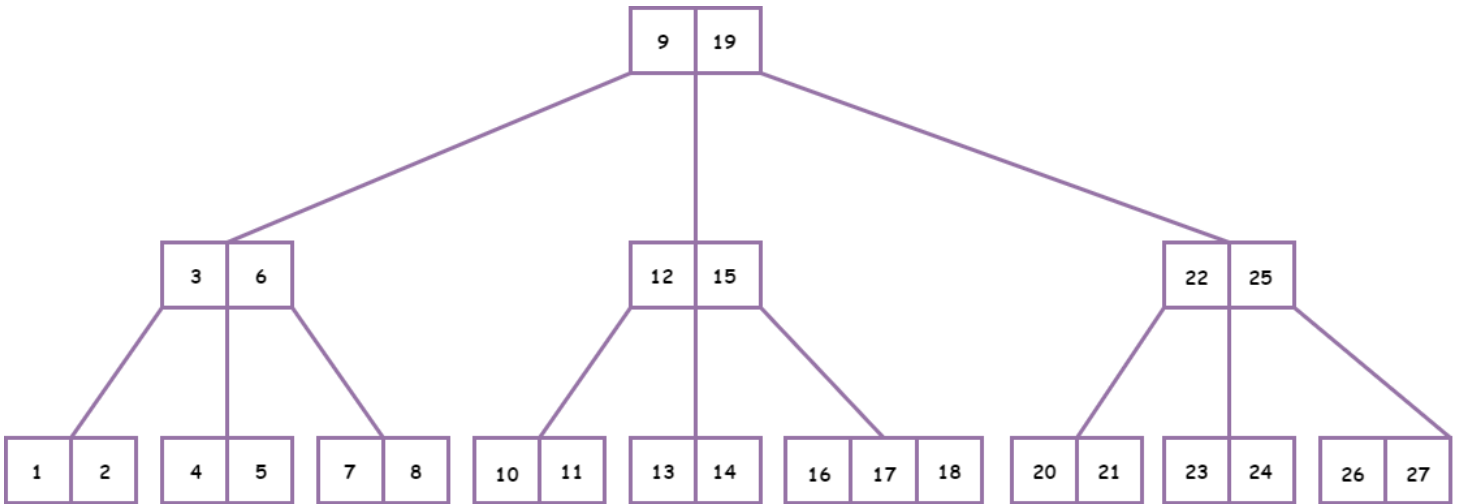
وهو عنصر داخلي أحد أبناء صفحته يملك أكثر من  $t$  عنصر سنقوم بتغييرات واستعارة من هذه الصفحة ثم نحذف العنصر: لدينا 7 هي العنصر التالي لل 6 وبالتالي سترفع ال 7 للأب ومن ثم نضيف ال 6 للصفحة المجاورة وهكذا أصبح بإمكاننا العودة للحالة الأولى واستبدال العنصر التالي مع العنصر المراد حذفه فنحذفه فتصبح:



**الحالة الرابعة:** جميع أبناء وأحفاد صفحة العنصر تحوي  $t$  عنصر فقط فسنقوم باستعارة عنصر من الأب لنضعه في المكان الذي نحتاجه ضمن ال Sub Tree الخاصة بالعنصر المراد حذفه وهكذا يمكننا إعادة الحالة لأحد الحالات الثلاث السابقة ونُعامل العنصر المأخوذ من الأب كأننا حذفنا عنصر فنعالجه أيضاً بإحدى الحالات الأربعة.

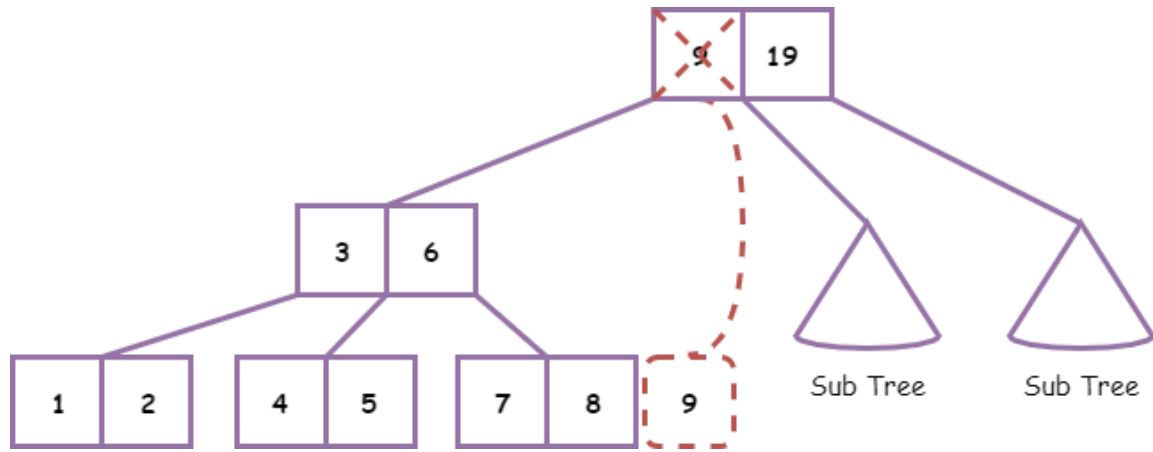
ملاحظة: عند وضع عنصر الأب في المكان المناسب فإنه سيكون ضمن ورقة إما في أقصى اليمين أو أقصى اليسار لأن الأب إما أن يكون أكبر من كل العناصر أو أصغر من كل العناصر

**مثال توضيحي:** لدينا B-Tree التالية حيث  $t = 2$ :

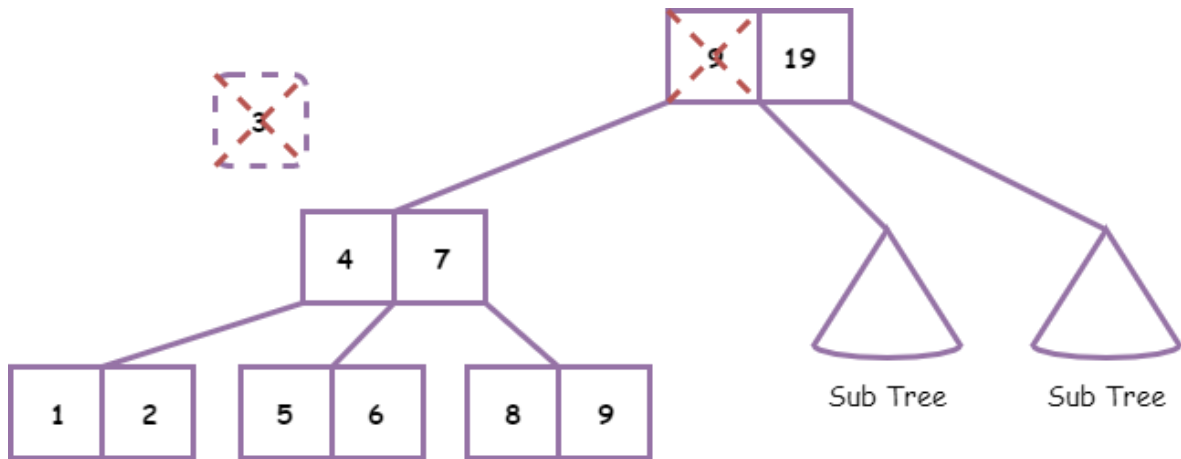


نريد حذف العنصر 3:

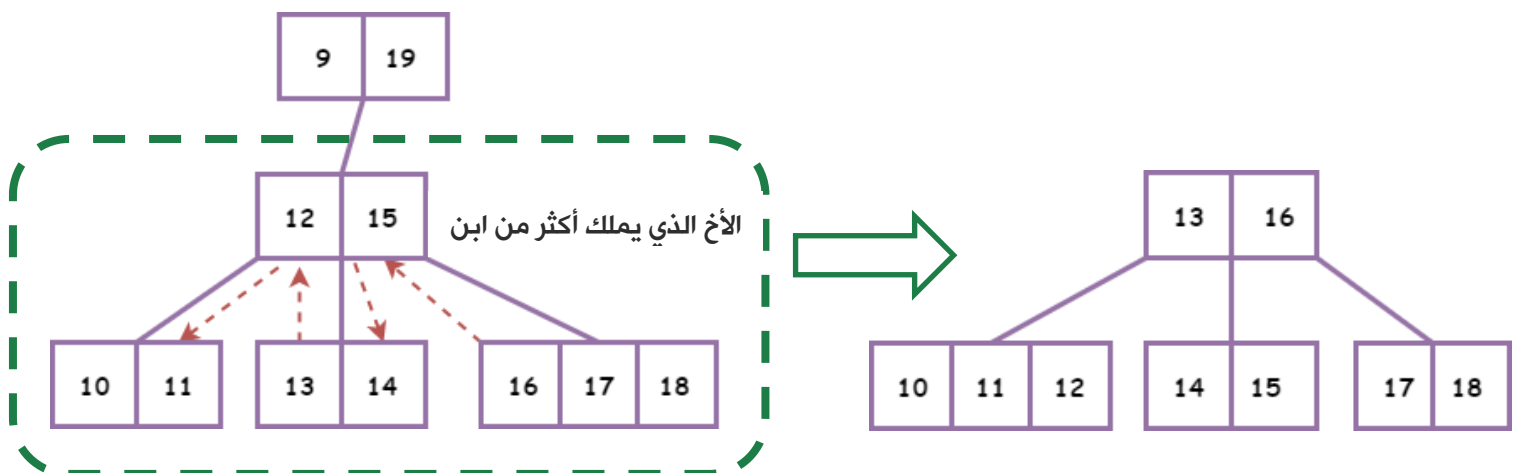
عنصر داخلي ولا نستطيع استبداله بالعنصر السابق أو التالي وكل أبنائه تحوي  $t$  عنصر وكل أبناء صفحته تحوي  $t$  عنصر ← حالة رابعة تتطلب عنصر من الأب ولناخذ 9 ونضعها في مكانها المناسب ضمن الصفحة التي تحوي العنصرين 7 و 8 ونضعها بأقصى اليمين.



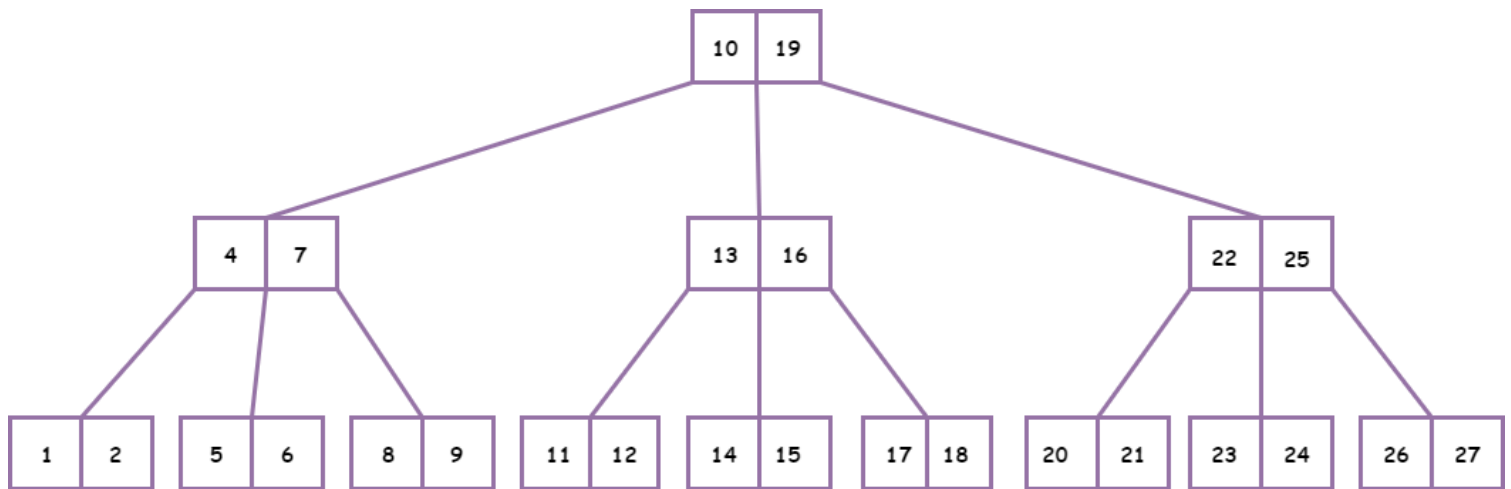
الآن عادت الشجرة لإحدى الحالات الثلاث الأولى: عنصر داخلي ابنها يملكان  $t$  عنصر واحد أبناء صفحتها يملك أكثر من  $t$  ← حالة ثالثة:



والآن تبقى لدينا معالجة حذف العنصر 9 من الجذ: نبحث عن العنصر التالي أو السابق وهما 10 و 8 على التوالي.  
مبدئياً العنصر السابق يحوي  $t$  عنصر ولكن أحد إخوته يحوي أكثر من  $t$ .



والآن أصبح العنصر التالي ل 9 هو 10 وهو قابل للحذف البسيط كما في الحالة الأولى



دون ملاحظتك

THE END ^\_^